Documento

# Manual Técnico

---




---

Escuela: Ing. E. Latzina.

Materia: Prácticas Profesionalizantes.

Docente: Serrano Aaron, Moya Juan.

Alumnos: Domínguez Victoria, Galíndez Sofía, Goncalves Luna, Iannopollo Angeline.

Curso: 6°2°, T.M, C.

Ciclo Lectivo: 2026.

**Tabla de Contenidos**


Sistema de Gestión de Notas ..... 5

1. Introducción ..... 5

Objetivo ..... 5

Alcance ..... 5

2. Arquitectura General ..... 5

Responsabilidades por Capa: ..... 6

3. Estructura del Proyecto ..... 6

Estructura del Backend (backend/) ..... 6

Estructura del Frontend (frontend/) ..... 7

4. Componentes y Módulos del Backend ..... 8

4.1 Controladores REST (com.sgn.ctrl) ..... 8

5. Documentación de API REST ..... 9

Módulo de Autenticación (/api/auth) ..... 9

6. Entidades JPA y Modelo Relacional ..... 10

Mapeo de Atributos Clave: ..... 10

7. Autenticación y Autorización ..... 11

Flujo de Autenticación JWT: ..... 11

Matriz de Permisos Basada en Roles (Spring Security + @PreAuthorize): ..... 11

8. Flujo Técnico de Primer Ingreso y Reseteo ..... 13

9. Componentes del Frontend (Next.js 14) ..... 13

Cliente HTTP Centralizado (frontend/lib/api.ts) ..... 13

Control de Rutas y Guards (frontend/lib/auth.ts) ..... 14

10. Gestión de Transacciones y Excepciones ..... 14

Transaccionalidad Backend (@Transactional) ..... 14

Manejador Global de Excepciones (@ControllerAdvice) ..... 14

11. Mantenimiento y Extensión del Sistema..... 14
Procedimiento para agregar un nuevo Endpoint / Módulo:..... 14

# Sistema de Gestión de Notas

## 1. Introducción

### Objetivo

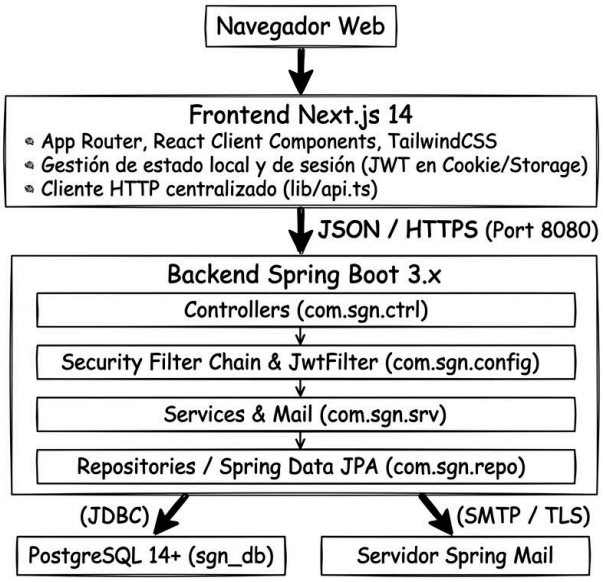
Este documento constituye la especificación técnica y de arquitectura del Sistema de Gestión de Notas (SGN) de la Escuela Ing. E. Latzina (ET35). Su propósito es proveer a desarrolladores, administradores de sistemas y mantenedores futuros una comprensión profunda de la estructura interna del código fuente, sus modelos de datos, patrones de diseño, mecanismos de seguridad y protocolos de integración.

### Alcance

El manual cubre la implementación real y verificada de los dos componentes principales que conforman la solución: el backend desarrollado en Java / Spring Boot y el frontend desarrollado en TypeScript / Next.js 14.

## 2. Arquitectura General

El SGN está implementado como una aplicación web desacoplada mediante una arquitectura de API REST N-Capas:




**Responsabilidades por Capa:**

- Frontend (Next.js 14): Encargado de la renderización del lado del cliente, maquetación adaptativa, validaciones de formularios en tiempo de ejecución, intercalación de token JWT en cabeceras HTTP y control de navegación mediante Guards (auth.ts).

- Backend (Spring Boot): Encargado de la lógica de negocio, validaciones de dominio, control de acceso basado en roles (@PreAuthorize), generación/verificación de tokens JWT, hasheado de claves (BCrypt), auditoría y envío diferido/asincrónico de notificaciones por correo electrónico (Spring Mail).

- Persistencia (PostgreSQL & Hibernate): Almacenamiento relacional de entidades de dominio. JPA/Hibernate gestiona la traducción ORM y la generación/actualización del esquema relacional.

### 3. Estructura del Proyecto

**Estructura del Backend (**backend/**)**

backend/

├── pom.xml

```
└── src/
    ├── main/
    │   ├── java/com/sgn/
    │   │   ├── config/      # Filtros JWT, SecurityFilterChain, CORS y Beans
    │   │   ├── ctrl/        # REST Controllers (Endpoints de la API)
    │   │   ├── dto/         # Data Transfer Objects (Requests y Responses)
    │   │   ├── enm/         # Enumeraciones de Dominio (Role, Estado, etc.)
    │   │   ├── ent/         # Entidades JPA cargadas en PostgreSQL
    │   │   ├── repo/        # Interfaces Spring Data JPA
    │   │   ├── srv/         # Servicios de Lógica de Negocio e Integración
    │   │   └── util/        # Generadores de tokens y utilitarios
    │   └── resources/
    │       └── application.yml
    └── test/                # Pruebas unitarias y de integración
```

**Estructura del Frontend (**frontend/**)**

```
frontend/
├── package.json
├── next.config.mjs
├── tsconfig.json
├── .env.local
```

```
└── app/                    # App Router de Next.js 14
    ├── (auth)/             # Rutas de autenticación (/login, /cambiar-password)
    ├── dashboard/          # Paneles por rol (/admin, /profesor, /alumno, etc.)
    ├── components/         # Componentes reutilizables UI
    └── lib/                # Cliente de API (api.ts) y helpers de sesión
```

## 4. Componentes y Módulos del Backend

### 4.1 Controladores REST (com.sgn.ctrl)

| Controller | Ruta Base | Responsabilidad Principal |
|---|---|---|
| AuthCtrl | /api/auth | Login, renovación de token, cambio obligatorio de clave y recovery. |
| UsuarioCtrl | /api/usuarios | CRUD de usuarios, reseteo administrativo de clave y conmutación de estado activo. |
| CursoCtrl | /api/cursos | Administración de cursos, divisiones, turnos y ciclos lectivos. |

| | | |
|---|---|---|
| MateriaCtrl | /api/materias | Mapeo y alta de asignaturas dentro de los planes de estudio. |
| BloqueCtrl | /api/bloques | Gestión de Bloques Académicos (asignación Docente-Materia-Curso). |
| NotaCtrl | /api/notas | Carga, edición y consulta de calificaciones parciales y finales. |
| AuditoriaCtrl | /api/auditoria | Consulta de logs de auditoría técnica y eventos del sistema. |

**5. Documentación de API REST**

**Módulo de Autenticación (**/api/auth**)**

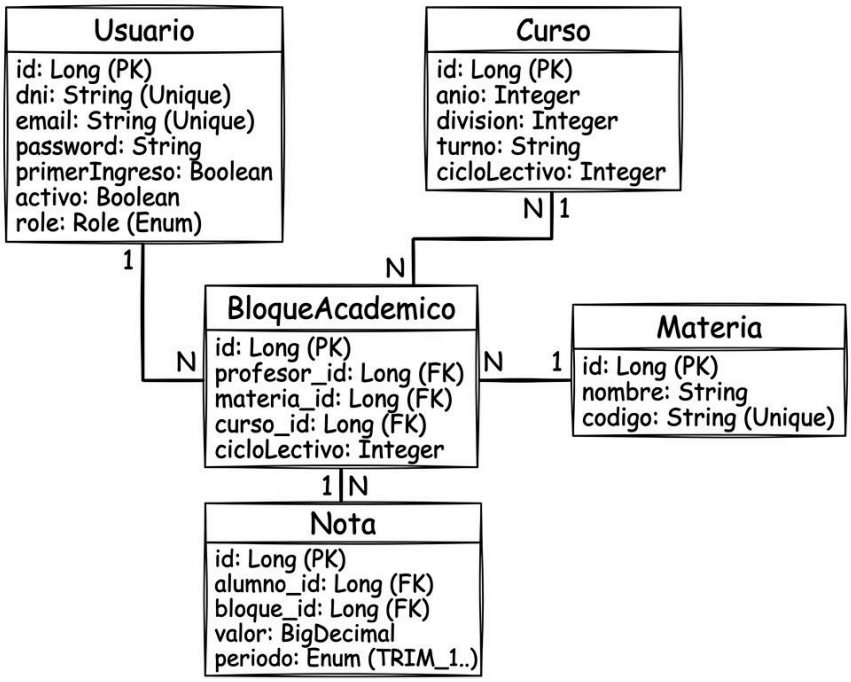
- POST /api/auth/login con acceso público. Su request body (LoginDTO) es: { "email": "usuario@et35.edu.ar", "password": "clave_ingresada" }. El Response Body (200 OK - AuthResponseDTO) es: { "token": "eyJhbGciOiJIUzUxMiJ9...", "email": "usuario@gmail.com", "role": "PROFESOR", "primerIngreso": false }. Puede tener código de error 401 Unauthorized (Credenciales inválidas o usuario inactivo).

- POST /api/auth/cambiar-password con acceso autenticado (primerIngreso = true o false). Su Request Body (CambiarPasswordDTO) es: { "passwordActual":

"clave_temporal", "nuevaPassword": "nueva_clave_segura" }. El Response Body (200 OK) es: { "mensaje": "Contraseña actualizada con éxito", "primerIngreso": false }.

### 6. Entidades JPA y Modelo Relacional

El modelo de datos se mapea relacionalmente sobre PostgreSQL mediante las siguientes entidades principales en com.sgn.ent:




**Mapeo de Atributos Clave:**

- Usuario**:** Mapea el estado de acceso de cada cuenta (primerIngreso, activo, role). Las contraseñas están encriptadas con BCryptPasswordEncoder.

- BloqueAcademico**:** Resuelve la relación N:M entre Usuario (Profesor), Materia y Curso para un ciclo escolar específico.

- Nota**:** Almacena la calificación asignada por un profesor a un alumno en un bloque académico determinado.

## 7. Autenticación y Autorización

**Flujo de Autenticación JWT:**

- El usuario envía credenciales a POST /api/auth/login.

- AuthSrv verifica el usuario en la base de datos a través de UsuarioRepo y compara el hash con BCrypt.checkpw().

- Si la verificación es exitosa y activo == true, JwtUtil genera una firma encriptada (algoritmo HMAC512) incorporando los claims: sub (email), role y primerIngreso.

- En cada petición subsecuente, la cabecera Authorization: Bearer <token> es interceptada por el filtro customizado JwtFilter (heredado de OncePerRequestFilter).

- JwtFilter valida la firma e inyecta la instancia de UsernamePasswordAuthenticationToken en el SecurityContextHolder de Spring Security.

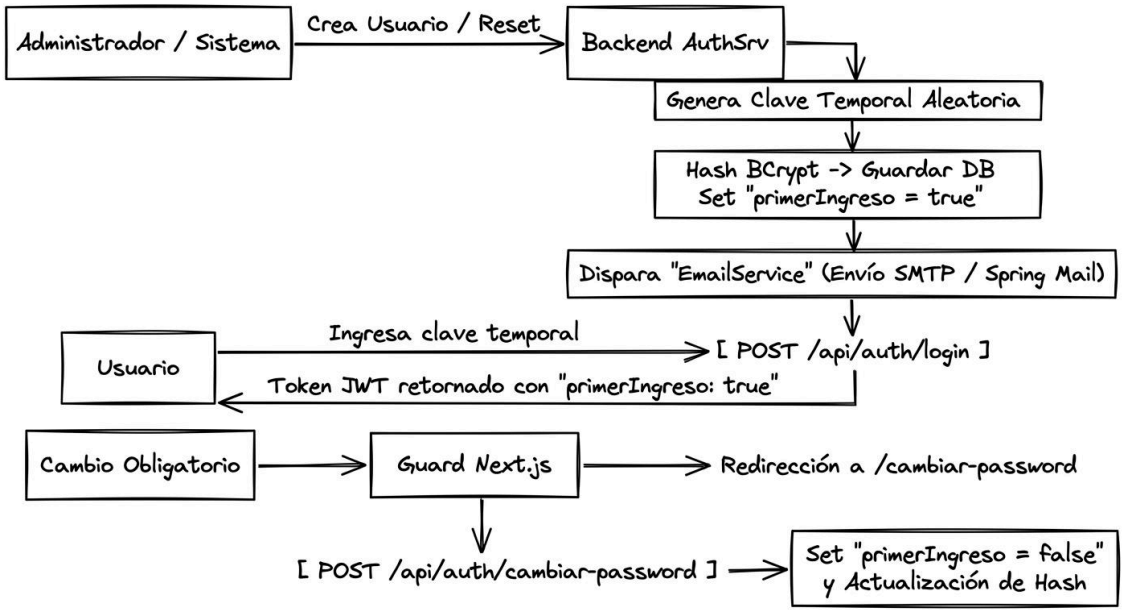
**Matriz de Permisos Basada en Roles (Spring Security +** @PreAuthorize**):**

| **Endpoint / Recurso** | **Admin** | **Oficina** | **Profesor** | **Alumno** | **Directivo / Preceptor** |
|---|---|---|---|---|---|
| /api/usuarios/** | RW | R | - | - | - |

| | | | | | |
|---|---|---|---|---|---|
| /api/usuarios/reset-password | RW | - | - | - | - |
| /api/cursos/** | RW | R | - | - | - |
| /api/materias/** | RW | R | - | - | - |
| /api/bloques/** | RW | - | - | - | - |
| /api/notas/cargar | - | - | RW | - | - |
| /api/notas/boletin/** | R | R | R | R (Propio) | R |
| /api/auditoria/** | RW | - | - | - | R |

RW = Lectura y Escritura, R = Solo Lectura, - = Sin Acceso.

## 8. Flujo Técnico de Primer Ingreso y Reseteo




## 9. Componentes del Frontend (Next.js 14)

**Cliente HTTP Centralizado (**frontend/lib/api.ts**)**

Toda comunicación con el backend se canaliza a través de un módulo centralizado que intercepta las peticiones:

- Inyección de Cabecera: Extrae el token JWT almacenado en las cookies/sessionStorage e inyecta la cabecera Authorization: Bearer <token>.

- Manejo de Respuestas: Detecta respuestas con código HTTP 401 Unauthorized o 403 Forbidden y destruye la sesión local, redirigiendo automáticamente al usuario al formulario de /login.

**Control de Rutas y Guards (**frontend/lib/auth.ts**)**

- Valida los permisos de navegación comparando el role del token del usuario con la ruta solicitada en el App Router.

- Intercepta la bandera primerIngreso == true en cualquier ruta del dashboard, forzando la redirección inmediata a /cambiar-password hasta que se complete la actualización de clave.

## 10. Gestión de Transacciones y Excepciones

**Transaccionalidad Backend (**@Transactional**)**

Los métodos críticos dentro de las clases de servicio en com.sgn.srv (como NotaSrv.guardarCalificaciones o UsuarioSrv.crearUsuario) están anotados con @Transactional(rollbackFor = Exception.class). Esto garantiza la atomicidad de las operaciones: si el servicio de correo o un repositorio secundario falla, se ejecuta un *rollback* completo de las inserciones en la base de datos.

**Manejador Global de Excepciones (**@ControllerAdvice**)**

Un componente global intercepta las excepciones producidas en los controladores y las mapea a respuestas JSON estandarizadas: ResourceNotFoundException -> HTTP 404 Not Found, BadCredentialsException -> HTTP 401 Unauthorized, AccessDeniedException -> HTTP 403 Forbidden y BusinessRuleException -> HTTP 400 Bad Request,

## 11. Mantenimiento y Extensión del Sistema

**Procedimiento para agregar un nuevo Endpoint / Módulo:**

- Definir la Entidad JPA: Crear la clase mapeada en com.sgn.ent con sus anotaciones JPA e identificador único @Id.

- Crear el Repositorio: Extender JpaRepository<Entidad, Long> en com.sgn.repo.

- Definir DTOs: Crear las estructuras Request/Response requeridas en com.sgn.dto.

- Implementar Servicio: Incluir la lógica de negocio y métodos de validación anotados con @Service y @Transactional en com.sgn.srv.

- Crear el Controlador REST: Definir los métodos HTTP (@GetMapping, @PostMapping, etc.) en com.sgn.ctrl asegurando la protección mediante anotaciones @PreAuthorize("hasRole('...')").

- Integrar en Frontend: Crear las llamadas API en frontend/lib/api.ts y maquetar la vista en el directorio frontend/app/ correspondiente al rol destinatario.